

量子模拟 ZMQ 服务器说明文档

OriginQ

2026 年 2 月 11 日

目录

1	项目简介	3
2	功能特性	3
3	系统架构	3
3.1	通信模式	4
3.1.1	1. 路由-代理模式（请求-响应）	4
3.1.2	2. 发布-订阅模式（发布-订阅）	4
4	安装指南	4
4.1	环境要求	4
4.2	Python 版本	4
5	快速开始	4
5.1	启动单个服务器	4
5.2	启动所有服务器	5
5.3	启动服务器（自动包含发布服务器）	5
5.4	自定义配置	5
5.5	命令行参数说明	6
6	协议详情	6
6.1	1. 超导量子系统	6
6.2	2. 离子阱量子系统	7
6.3	3. 中性原子量子系统	8
6.4	4. 光量子系统	8

目录	2
7 任务流程	9
7.1 1. 提交任务（路由）	9
7.2 2. 接收确认（路由）	9
7.3 3. 任务状态更新（发布）	9
7.4 4. 接收结果（路由）	10
8 配置说明	10
8.1 服务器配置 (config.py)	10
9 任务状态枚举与错误码	11
9.1 任务状态枚举	11
9.2 错误码	12
10 客户端示例	12
10.1 路由-代理客户端示例	12
10.2 发布-订阅客户端示例	13
11 测试	13
11.1 启动所有服务器	13
11.2 使用测试客户端	13
12 日志	14
13 常见问题排查	14
13.1 端口已被占用	14
13.2 响应中缺少 SN 字段	14
13.3 Token 刷新失败（离子阱和中性原子）	14
13.4 任务状态未发布	15
14 发布-订阅消息结构	15
15 许可证	15
16 技术支持	15

1 项目简介

Python 实现的量子计算模拟服务，支持四种量子系统：超导、离子阱、中性原子和光量子，用于联合本源司南操作系统使用；如需使用本服务，可参考司南图形界面中添加芯片功能，完成对应芯片信息配置后，即可使用。

2 功能特性

- **ZMQ 路由-代理模式**：处理客户端请求并发送响应
- **ZMQ 发布-订阅模式**：实时推送状态更新和通知
- **多协议支持**：支持四种不同的量子系统协议
- **任务管理**：异步任务处理，支持状态跟踪
- **状态发布**：通过发布-订阅自动推送任务状态更新
- **随机结果生成**：模拟量子计算结果
- **端口范围 7000-7010**：路由端口用于客户端请求
- **端口范围 8000-8010**：发布端口用于状态更新

3 系统架构

```
1 PilotPy/python_simulator/  
2     config.py                # 配置和枚举定义  
3     result_generator.py      # 随机结果生成器  
4     task_manager.py          # 任务生命周期管理  
5     zmq_router_server.py     # 核心ZMQ路由服务器  
6     zmq_pub_server.py        # ZMQ发布服务器（状态更新）  
7     main.py                  # 程序入口  
8     protocol_adapters/  
9         superconducting.py   # 超导协议适配器  
10        ion_trap.py           # 离子阱协议适配器  
11        neutral_atom.py       # 中性原子协议适配器  
12        photonic.py           # 光量子协议适配器  
13    README_CN.md              # 本文件（中文版说明）
```

文件结构

3.1 通信模式

服务器使用两种互补的 ZMQ 通信模式：

3.1.1 1. 路由-代理模式（请求-响应）

- **端口：**7000-7003
- **用途：**处理客户端请求并发送响应
- **连接方式：**客户端作为 DEALER 连接，服务器作为 ROUTER
- **通信特点：**双向通信

3.1.2 2. 发布-订阅模式（发布-订阅）

- **端口：**8000-8003
- **用途：**向订阅者推送状态更新和通知
- **连接方式：**服务器作为 PUBLISHER，客户端作为 SUBSCRIBER
- **通信特点：**单向推送通信，实时状态更新
- **消息结构：**三层消息结构（主题 + 操作 + 数据）

4 安装指南

4.1 环境要求

```
1 pip install pyzmq
```

4.2 Python 版本

- Python 3.8 或更高版本

5 快速开始

5.1 启动单个服务器

启动特定的量子系统服务器：

```
1 # 启动超导量子服务器（默认端口7000）
2 python main.py --system superconducting
3
4 # 启动离子阱量子服务器（默认端口7001）
5 python main.py --system ion_trap
6
7 # 启动中性原子量子服务器（默认端口7002）
8 python main.py --system neutral_atom
9
10 # 启动光量子服务器（默认端口7003）
11 python main.py --system photonic
```

5.2 启动所有服务器

同时启动所有四个量子系统服务器：

```
1 python main.py --all
```

5.3 启动服务器（自动包含发布服务器）

启动一个服务器时，会自动同时启动路由服务器和发布服务器：

```
1 python main.py --system superconducting
```

输出示例：

```
1 =====
2 Starting superconducting Quantum Simulation Server
3 =====
4
5 superconducting server is running
6   - Router Port: 7000
7   - Pub Port: 8000
8   - Bind Address: 0.0.0.0
9   - Log Level: INFO
10  - Thread Pool Size: 4
11  - Max Queue Size: 1000
12
13 Press Ctrl+C to stop server
```

5.4 自定义配置

```
1 # 自定义端口（仅影响路由端口）
2 python main.py --system superconducting --port 7005
3
4 # 自定义绑定地址
5 python main.py --system superconducting --bind-address 192.168.1.100
6
7 # 设置日志级别
8 python main.py --system superconducting --log-level DEBUG
```

5.5 命令行参数说明

```
1 --system {superconducting,ion_trap,neutral_atom,photonic}
2                               要模拟的量子系统类型
3 --all                          启动所有量子系统服务器
4 --port PORT                    覆盖默认端口
5 --log-level {DEBUG,INFO,WARNING,ERROR}
6                               设置日志级别（默认：INFO）
7 --bind-address ADDRESS        绑定地址（默认：*）
```

6 协议详情

6.1 1. 超导量子系统

- 路由端口：7000
- 发布端口：8000

支持的消息类型（路由）：

- MsgTask：提交量子计算任务
- TaskStatus：查询任务状态
- MsgHeartbeat：心跳检测
- GetChipConfig：获取芯片配置
- GetUpdateTime：获取校准时间
- GetRBData：获取随机基准测试数据
- SetVip：设置独占时间片

- ReleaseVip: 释放独占时间片

发布的消息类型 (发布):

- task_status: 任务状态更新 (PENDING, RUNNING, SUCCEEDED, FAILED)
- chip_update: 芯片配置更新通知
- probe: 芯片资源状态 (量子比特使用情况、线程状态)
- calibration_start: 校准开始通知
- calibration_done: 校准完成通知
- chip_protect: 芯片维护开始/结束通知

协议特点: 扁平 JSON 结构, 支持任务优先级、实验模式、VIP 时间片管理及实时状态更新。

6.2 2. 离子阱量子系统

- 路由端口: 7001
- 发布端口: 8001

支持的消息类型 (路由):

- MsgGetToken: 获取访问令牌 (认证)
- MsgUpdateToken: 刷新访问令牌
- MsgTask: 提交量子计算任务
- TaskStatus: 查询任务状态
- MsgHeartbeat: 心跳检测
- GetChipConfig: 获取芯片配置
- GetUpdateTime: 获取校准时间
- GetRBData: 获取随机基准测试数据

发布的消息类型 (发布): task_status

协议特点: Header/Body JSON 结构, 基于 Token 的认证机制, 支持版本号字段和保真度矩阵。

6.3 3. 中性原子量子系统

- 路由端口：7002
- 发布端口：8002

支持的消息类型（路由）：

- MsgGetToken：获取访问令牌（认证）
- MsgTask：提交量子计算任务
- MsgTaskStatus：查询任务状态
- MsgHeartbeat：心跳检测
- GetUpdateTime：获取校准时间
- MsgAtomConfig：获取原子配置

发布的消息类型（发布）：`task_status`

协议特点：Header/Body JSON 结构，基于 Token 的认证机制，OPENQASM 任务格式，自定义结果格式（包含 grid 和 waveform）。

6.4 4. 光量子系统

- 路由端口：7003
- 发布端口：8003

支持的消息类型（路由）：

- MsgTask：提交量子计算任务
- TaskStatus：查询任务状态
- MsgHeartbeat：心跳检测
- GetChipConfig：获取芯片配置

发布的消息类型（发布）：`task_status`

协议特点：扁平 JSON 结构，支持基本量子门，QASM 风格任务格式。

7 任务流程

7.1 1. 提交任务（路由）

```
1 {  
2     "MsgType": "MsgTask",  
3     "SN": 1,  
4     "TaskId": "unique-task-id",  
5     "ConvertQProg": "...",  
6     "Configure": {  
7         "Shot": 1000  
8     }  
9 }
```

7.2 2. 接收确认（路由）

```
1 {  
2     "MsgType": "MsgTaskAck",  
3     "SN": 1,  
4     "ErrCode": 0,  
5     "ErrInfo": ""  
6 }
```

7.3 3. 任务状态更新（发布）

任务状态通过发布-订阅自动推送：

PENDING（接收任务后）：

```
1 {  
2     "MsgType": "TaskStatus",  
3     "SN": 0,  
4     "TaskId": "unique-task-id",  
5     "TaskStatus": 1  
6 }
```

RUNNING（处理中）：

```
1 {  
2     "MsgType": "TaskStatus",  
3     "SN": 0,  
4     "TaskId": "unique-task-id",  
5     "TaskStatus": 2  
6 }
```

SUCCEEDED (完成后):

```
1 {
2     "MsgType": "TaskStatus",
3     "SN": 0,
4     "TaskId": "unique-task-id",
5     "TaskStatus": 5
6 }
```

7.4 4. 接收结果 (路由)

```
1 {
2     "MsgType": "MsgTaskResult",
3     "SN": 1,
4     "TaskId": "unique-task-id",
5     "Key": [["0x0", "0x1"]],
6     "ProbCount": [[500, 500]],
7     "NoteTime": {
8         "CompileTime": 100,
9         "MeasureTime": 2000,
10        "PostProcessTime": 50
11    }
12 }
```

8 配置说明

8.1 服务器配置 (config.py)

```
1 class ServerConfig:
2     # 网络设置
3     BIND_ADDRESS = "*"           # 绑定到所有网络接口
4     TIMEOUT = 1000              # Socket超时时间 (毫秒)
5     HWM = 1000                  # 高水位标记
6
7     # 路由端口分配
8     ROUTER_PORTS = {
9         QuantumSystemType.SUPERCONDUCTING: 7000,
10        QuantumSystemType.ION_TRAP: 7001,
11        QuantumSystemType.NEUTRAL_ATOM: 7002,
12        QuantumSystemType.PHOTONIC: 7003
13    }
14
15     # 发布端口分配
```

```

16     PUB_PORTS = {
17         QuantumSystemType.SUPERCONDUCTING: 8000,
18         QuantumSystemType.ION_TRAP: 8001,
19         QuantumSystemType.NEUTRAL_ATOM: 8002,
20         QuantumSystemType.PHOTONIC: 8003
21     }
22
23     # 性能设置
24     THREAD_POOL_SIZE = 10          # 并发任务工作线程数
25     MAX_QUEUE_SIZE = 200          # 最大待处理任务数
26
27     # 模拟时间 (毫秒)
28     COMPILE_TIME = 100            # 模拟编译时间
29     RUN_TIME = 2000               # 模拟运行时间
30     POST_PROCESS_TIME = 50        # 模拟后处理时间
31
32     # 日志设置
33     LOG_LEVEL = "INFO"
34     LOG_FILE = "log/simulator.log"

```

9 任务状态枚举与错误码

9.1 任务状态枚举

```

1 class TaskStatus(Enum):
2     UNKNOWN_STATE = 0          # 未知状态
3     PENDING = 1                # 任务在队列中
4     COMPILING = 7              # 任务编译中
5     COMPILED = 8               # 任务已编译
6     RUNNING = 2                # 任务运行中
7     SUCCEEDED = 5              # 任务成功完成
8     FAILED = 4                 # 任务失败
9     NOTASK = 3                 # 任务不存在
10    RETRY = 6                   # 任务重试
11
12    # 中性原子专用
13    SUBMIT = 9                  # 任务已提交
14    FINISH = 10                 # 任务已完成
15    CANCEL = 11                 # 任务已取消
16    SUBMITFAIL = 12             # 提交失败
17    RUNFAIL = 13                # 运行失败
18    WAITING = 14                # 任务等待中

```

9.2 错误码

```

1 class ErrorCode(Enum):
2     NO_ERROR = 0           # 成功
3     UNDEFINED_ERROR = 1    # 未知错误
4     TASK_PARAM_ERROR = 2   # 任务参数错误
5     JSON_ERROR = 3         # JSON解析错误
6     QUEUE_FULL = 4         # 队列已满
7     AUTH_ERROR = 5         # 认证错误
8     TASK_ID_DUPLICATE = 10  # 任务ID重复
9     TASK_ID_NOT_EXIST = 40  # 任务ID不存在

```

10 客户端示例

10.1 路由-代理客户端示例

```

1 import zmq
2 import json
3
4 # 连接到路由服务器
5 context = zmq.Context()
6 socket = context.socket(zmq.DEALER)
7 socket.connect("tcp://localhost:7000")
8
9 # 提交任务
10 task = {
11     "MsgType": "MsgTask",
12     "SN": 1,
13     "TaskId": "test-task-001",
14     "ConvertQProg": "[[{'H': [0]}, {'Measure': [[0]]}]]",
15     "Configure": {"Shot": 1000}
16 }
17
18 # 发送请求
19 socket.send_json(task)
20
21 # 接收确认
22 ack = socket.recv_json()
23 print(f"确认消息: {ack}")
24
25 # 接收结果 (异步)
26 result = socket.recv_json()
27 print(f"任务结果: {result}")

```

10.2 发布-订阅客户端示例

```
1 import zmq
2 import json
3
4 # 连接到发布服务器
5 context = zmq.Context()
6 socket = context.socket(zmq.SUB)
7 socket.connect("tcp://localhost:8000")
8
9 # 订阅主题 'simulator_topic' 以接收所有消息
10 socket.setsockopt_string(zmq.SUBSCRIBE, b'simulator_topic')
11
12 # 接收状态更新
13 while True:
14     # 接收主题
15     topic = socket.recv()
16
17     # 接收操作类型
18     operation = socket.recv()
19
20     # 接收数据
21     data = socket.recv_json()
22
23     print(f"接收消息: topic={topic}, operation={operation}")
24     print(f"数据: {data}")
```

11 测试

11.1 启动所有服务器

```
1 python main.py --all
```

11.2 使用测试客户端

```
1 python test_client.py
```

测试特定服务器:

```
1 # 测试超导服务器
2 python test_client.py --port 7000
3
4 # 测试离子阱服务器
```

```
5 python test_client.py --port 7001
6
7 # 测试所有服务器
8 python test_client.py --all
```

12 日志

日志存储在 log/ 目录中。日志级别包括：

- **DEBUG**：详细的调试信息
- **INFO**：一般信息消息（默认）
- **WARNING**：警告消息
- **ERROR**：仅错误消息

13 常见问题排查

13.1 端口已被占用

如果看到 Address already in use 错误：

```
1 # 查找使用端口的进程
2 lsof -i :7000
3 lsof -i :8000
4
5 # 终止进程
6 kill -9 <PID>
```

13.2 响应中缺少 SN 字段

问题：响应消息不包含 SN 或 SN 值不正确。

解决方案：确保使用最新代码：

```
1 git pull # 获取最新更改
2 python main.py --all # 重启服务器
```

13.3 Token 刷新失败（离子阱和中性原子）

问题：MsgUpdateToken 返回空或无效的令牌。

解决方案：1. 确保在 Authorization 头中发送 RefreshToken。2. 检查刷新令牌是否之前从 MsgGetToken 获得。3. 验证令牌未过期。

13.4 任务状态未发布

问题：客户端未通过发布-订阅接收到任务状态更新。

解决方案：1. 验证发布服务器正在运行（检查端口 8000-8003）。2. 确保订阅者订阅了正确的主题。3. 检查防火墙是否允许发布服务器端口。

14 发布-订阅消息结构

发布服务器使用三层消息结构：

1. **主题：**所有消息固定为 `b'simulator_topic'`
2. **操作：**消息类型（如 `b'task_status'`, `b'chip_update'`）
3. **数据：**JSON 数据载荷

15 许可证

这是一个用于开发和测试目的的模拟服务器。

16 技术支持

如有问题或疑问，请参考协议文档：

- 超导：超导通信协议文档.md
- 离子阱：离子阱通信协议文档.md
- 中性原子：中性原子通信协议文档.md
- 光量子：光量子通信协议文档.md